

API Testing From Zero to Viva-Ready

Everything explained in simple words — API basics, HTTP, API testing, Postman, Newman, CI/CD + 45 viva questions with answers

Quick-scan guide for SQA interviews and vivas

Includes a cheat-sheet for your project:
api-test-automation (Restful-Booker · Postman · Newman · GitHub Actions)

PART 1 — API Basics (start here)

1.1 What is an API? (the waiter story)

You sit in a restaurant. You don't walk into the kitchen — you tell the **waiter** what you want, the waiter takes your order to the kitchen, and brings your food back. An **API (Application Programming Interface)** is that waiter. Your app (the customer) asks for something, the API carries the request to the server (the kitchen), and brings back the response (the food). You never see the kitchen — you only care that you asked correctly and got the right food back.

Real example: When a weather app shows today's temperature, it doesn't have a thermometer. It calls a weather API: 'give me the weather for Dhaka' — and the server sends back the data.

1.2 Request and Response — the whole game

Every API interaction has exactly two halves:

	What it is	Made of
REQUEST	What the client sends TO the server	Method + URL + Headers + Body (sometimes)
RESPONSE	What the server sends BACK	Status code + Headers + Body (usually JSON)

API testing in one sentence: **send a request, then check that the response is exactly what it should be.**

1.3 HTTP Methods (the verbs)

The method tells the server what KIND of action you want. Think CRUD: Create, Read, Update, Delete.

Method	Means	Everyday example
GET	Read / fetch data (changes nothing)	Open your Facebook profile
POST	Create something new	Post a new status
PUT	Replace something completely (full update)	Re-write your whole bio
PATCH	Change only part of something	Fix one word in your bio
DELETE	Remove something	Delete a post

Viva favourite — **PUT vs PATCH:** PUT sends the FULL object and replaces everything; PATCH sends ONLY the fields you want to change. In my project I proved this: after PATCH-ing only firstname, I asserted that lastname and price were still unchanged.

1.4 Status Codes (the server's one-line reply)

The first thing you check in any test. Memorize the families: **2xx = success**, **4xx = client's fault**, **5xx = server's fault**.

Code	Name	Simple meaning
200	OK	Done, here's your data
201	Created	New thing created successfully
204	No Content	Done, nothing to send back (common for DELETE)
400	Bad Request	Your request is broken (bad JSON, missing field)
401	Unauthorized	Who are you? Log in first (no/invalid credentials)
403	Forbidden	I know who you are — you're still not allowed

Code	Name	Simple meaning
404	Not Found	That thing doesn't exist
500	Internal Server Error	Server crashed — always a bug on their side
503	Service Unavailable	Server is down / overloaded

Viva favourite — **401 vs 403**: 401 = you didn't prove who you are (bad/missing login). 403 = identity is fine, but you don't have permission. Bouncer analogy: 401 = no ID card; 403 = valid ID, but you're not on the guest list.

1.5 Endpoint, Headers, Body, Parameters

Term	Simple meaning	Example
Endpoint	The URL address of one specific function	/booking/971 = booking number 971
Base URL	The fixed front part of every endpoint	https://restful-booker.herokuapp.com
Header	Extra info label on the request/response	Content-Type: application/json
Body / Payload	The actual data you send (POST/PUT/PATCH)	{ "firstname": "Abrar" }
Path parameter	Value inside the URL path	/booking/{id} — the id part
Query parameter	Filter after the ? in the URL	/booking?firstname=Sally

1.6 JSON — the language of APIs

JSON (JavaScript Object Notation) is how request and response data is written. It's just **key : value pairs** inside curly braces. Human-readable, machine-parseable.

```
{ "firstname": "Abrar", "totalprice": 250, "depositpaid": true,
  "bookingdates": { "checkin": "2026-08-01", "checkout": "2026-08-07" } }
```

Value types: string ("text"), number (250), boolean (true/false), object ({...}), array ([...]), null.

1.7 What is REST?

REST is a set of design rules for APIs: use HTTP methods for CRUD, use URLs to name resources (nouns like /booking, not verbs), be **stateless** (every request carries everything the server needs — the server remembers nothing between requests), and usually speak JSON. An API following these rules is called a **RESTful API**. The alternative, SOAP, is an older, heavier XML-based protocol.

PART 2 — API Testing

2.1 What is API testing and why do we do it?

API testing means testing the application's logic layer **directly** — sending requests and validating responses — without any UI. Why it matters:

- **Faster** — no browser to load; my 15-request suite runs in ~7 seconds.
- **Earlier** — you can test the backend before the UI even exists (shift-left).
- **More stable** — no flaky buttons/locators like UI automation.
- **Deeper** — you can hit error cases the UI would never let you create.

2.2 The 5 things you check in EVERY response

#	Check	Example from my project
1	Status code	POST /booking must return 200
2	Response body values	firstname in response = firstname I sent
3	Response structure (schema)	bookingid must exist and be an integer
4	Headers	Content-Type must be application/json
5	Response time	Every request under 5000 ms SLA

2.3 Types of API testing

Type	Question it answers	In my project
Smoke	Is the API even alive?	GET /ping gates the whole suite
Functional	Does each endpoint do its job?	Full CRUD lifecycle on /booking
Negative	Does it fail correctly on bad input?	Malformed JSON must NOT create a booking
Boundary	Does it handle edge values?	Non-existent ID 999999999 returns 404
Security (basic)	Are protected actions really protected?	PUT/DELETE without token must return 403
Contract / Schema	Is the response structure stable?	JSON Schema validation on create
Performance check	Is it fast enough?	Response-time assertion on every request
Regression	Did new code break old features?	Whole suite re-runs in CI on every push

2.4 Positive vs Negative testing

Positive: give valid input, expect success (create booking with correct data → 200 + booking created).

Negative: give invalid input, expect the RIGHT failure (wrong password → no token; broken JSON → 400, never 200). A good tester spends as much energy proving the API fails safely as proving it succeeds.

2.5 Test design techniques (name-drop these in viva)

- **Equivalence partitioning:** group inputs into classes — valid credentials vs invalid credentials; one test per class.
- **Boundary value analysis:** test the edges — price 1 and 9999, a huge non-existent ID.
- **State transition:** test the lifecycle order — Create → Read → Update → Delete → verify 404.
- **Error guessing:** use experience to attack — malformed JSON, missing token.

2.6 What is a Test Plan / Test Case / Bug Report?

- **Test plan:** the strategy document — scope (what will/won't be tested), approach, entry/exit criteria, environment, risks, schedule.
- **Test case:** one specific check — ID, title, preconditions, steps, expected result, priority. My repo has 16, each mapped to an automated request.
- **Bug report:** when actual $\neq$ expected — title, steps to reproduce, expected vs actual, severity/priority, evidence.
- **Severity vs Priority:** severity = how badly it damages the system (technical); priority = how fast it must be fixed (business). A typo in a legal disclaimer can be low severity but high priority.

PART 3 — Postman, Newman & Automation

3.1 Postman building blocks

Concept	Simple meaning
Collection	A folder of saved requests + their tests. My whole suite = one collection with 5 folders.
Environment	A set of variables for one setup (QA/staging/prod). Switch environment = same tests, different server.
Variable	A named placeholder like <code>{{baseUrl}}</code> or <code>{{authToken}}</code> used inside requests.
Pre-request script	JavaScript that runs BEFORE the request is sent. I use it to generate random test data.
Test script	JavaScript that runs AFTER the response arrives. This is where assertions live.
Collection Runner	Runs every request in the collection in order, showing pass/fail.

3.2 How a test is written (the actual code)

Postman tests are JavaScript using the Chai assertion library:

```
pm.test('Status code is 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Token exists', function () {
  const json = pm.response.json();
  pm.expect(json).to.have.property('token');
});
```

Read it in plain English: 'I test that the status is 200' and 'I expect the JSON to have a token property'. That's all an assertion is.

3.3 Request chaining (how my token flows)

Real workflows need data from one request inside the next. Steps in my project:

```
1. POST /auth returns a token → my test script saves it:
   pm.collectionVariables.set('authToken', json.token);
2. POST /booking returns a bookingid → saved the same way.
3. Later requests use {{authToken}} and {{bookingId}} in their URL/headers.
Result: I test the real user journey, not isolated calls.
```

3.4 Authentication & tokens

A **token** is like a movie ticket: show your ID once at the counter (username+password → POST /auth), get a ticket (token string), then show only the ticket at every door (Cookie: token=abc123 on PUT/PATCH/DELETE). Safer than sending the password every time, and tokens can expire. Other auth types worth naming: **Basic Auth** (username:password encoded in a header), **Bearer token / OAuth 2.0** (Authorization: Bearer <token> — the industry standard), **API key** (a fixed secret string).

3.5 What is Newman?

Newman = Postman's command-line runner. Postman is the GUI where I build and debug; Newman runs the SAME collection file from the terminal with one command — no clicking, no human needed. That is what makes CI possible. It also generates reports (I use the `htmlextra` reporter for a shareable HTML report).

```
newman run collection.json -e environment.json -r "cli,htmlextra"
```

3.6 CI/CD with GitHub Actions

CI (Continuous Integration) = automatically testing every code change. My repo has a workflow file (`.github/workflows/api-tests.yml`) that tells GitHub: on every push, on every pull request, every night at 2 AM UTC, and on manual demand — spin up a Linux machine, install Newman, run the full suite, publish the results and upload the HTML report. If anything fails, the commit gets a red X. My tests became a **regression safety net** that never sleeps.

3.7 Data-driven testing

Instead of one hard-coded input, feed a CSV file of many rows — Newman runs the requests once per row. My `data/booking-data.csv` has 5 rows covering boundaries (price 1, price 9999). One request design, five test iterations. I also generate random names/prices in pre-request scripts so every run is independent and repeatable.

PART 4 — Advanced Concepts (short & sharp)

JSON Schema / Contract testing

A schema is a blueprint of the response: which fields must exist and their types.

`pm.response.to.have.jsonSchema(schema)` fails if a developer silently changes `bookingid` from integer to string — status 200 alone would never catch that. The 'contract' is the agreement between API producer and consumer.

Idempotency

An operation is idempotent if doing it many times has the same effect as doing it once. GET, PUT, DELETE are idempotent; POST is not (each POST creates a NEW booking). Classic viva question.

Statelessness

REST servers remember nothing between requests — every request must carry everything needed (like the token). That's why I re-send the token on every mutation.

REST vs SOAP

REST: architectural style, lightweight, JSON, uses HTTP methods. SOAP: strict protocol, XML envelopes, heavier, built-in standards (WS-Security). Modern web = mostly REST.

Rate limiting / 429

Servers limit how many requests you may send per minute; exceeding it returns 429 Too Many Requests. Why my CI adds a small delay between requests.

Mock server

A fake API that returns canned responses. Lets QA write tests before the real backend exists, and test error cases (like forced 500s) that are hard to trigger on a real server. Postman can create mock servers.

API documentation (Swagger/OpenAPI)

A machine-readable description of every endpoint, parameter, and response. As a tester it's your source of truth for expected behavior. Restful-Booker publishes `apidoc` — I tested against it.

Cookie vs Header auth

Restful-Booker expects the token in a Cookie header (Cookie: token=abc). Many APIs use Authorization: Bearer abc instead. Same idea, different envelope — always read the docs.

Environment strategy

Never hard-code URLs or credentials in requests. Keep them in environment files so one collection runs against QA, staging, or prod by swapping a file. Secrets stay out of the collection (and out of git for real projects).

Flaky tests & test independence

A flaky test passes/fails randomly. My defenses: each run creates its own data (no dependence on leftover records), generous SLAs for a shared public server, and a smoke test to warm up cold starts.

PART 5 — 45 Viva Questions with Answers

Rapid-fire format. Say answers out loud until they sound like yours.

A. API Fundamentals

Q: What is an API?

A: A messenger that lets two software systems talk over HTTP. The client sends a request; the server returns a response. Like a waiter between the customer and the kitchen.

Q: What is a REST API?

A: An API following REST rules: HTTP methods for CRUD, resource-based URLs, stateless requests, usually JSON.

Q: What are the main HTTP methods?

A: GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove).

Q: Difference between PUT and PATCH?

A: PUT replaces the whole resource — send everything. PATCH changes only the fields you send. I verified PATCH semantics by asserting untouched fields stayed the same.

Q: Difference between GET and POST?

A: GET reads data, has no body, parameters go in the URL, is safe to repeat. POST creates data, carries a body, and creates something new each time.

Q: What is an endpoint?

A: The specific URL for one API function, e.g. /booking/971 is the endpoint for booking 971.

Q: What is a payload?

A: The data (body) sent with a request — usually JSON — e.g. the booking details in my POST /booking.

Q: What are status code families?

A: 1xx info, 2xx success, 3xx redirect, 4xx client error, 5xx server error.

Q: 401 vs 403?

A: 401 Unauthorized = not logged in / bad credentials. 403 Forbidden = logged in but not allowed. No ID card vs valid ID but not on the guest list.

Q: What is JSON?

A: A lightweight key:value data format APIs use to exchange data. Human-readable, easy to parse.

Q: What is a header? Give an example.

A: Metadata attached to a request/response. Content-Type: application/json tells the server the body is JSON.

Q: Path parameter vs query parameter?

A: Path param is part of the URL path (/booking/971 — identifies WHICH resource). Query param filters after the ? (/booking?firstname=Sally — HOW to filter).

Q: What is statelessness in REST?

A: The server keeps no memory between requests; each request must carry everything needed, including auth.

Q: What is idempotency?

A: Repeating the operation gives the same result as doing it once. GET/PUT/DELETE are idempotent; POST is not.

Q: REST vs SOAP?

A: REST: style, light, JSON over HTTP. SOAP: strict protocol, XML, heavier, older enterprise standard.

B. API Testing

Q: What is API testing?

A: Testing the business-logic layer directly by sending requests and validating responses — no UI. Faster, earlier, and more stable than UI testing.

Q: What do you validate in a response?

A: Five things: status code, body values, structure/schema, headers, and response time.

Q: Why API testing over UI testing?

A: It runs in seconds, starts before the UI exists (shift-left), avoids flaky locators, and reaches error cases the UI can't produce. They complement each other — API tests the logic, UI tests the experience.

Q: What is negative testing? Example?

A: Proving the API fails CORRECTLY on bad input. Example from my project: malformed JSON to POST /booking must return 4xx, never 200; wrong credentials must not return a token.

Q: What is smoke testing in APIs?

A: A quick 'is it alive?' check before running everything. My GET /ping gates the whole suite.

Q: What is regression testing?

A: Re-running existing tests after changes to confirm nothing old broke. My whole collection re-runs in CI on every push — that IS my regression suite.

Q: What is contract/schema testing?

A: Validating the response STRUCTURE against a JSON Schema, so a silent field-type change fails the build even when the status is still 200.

Q: What test design techniques did you use?

A: Equivalence partitioning (valid vs invalid credentials), boundary value analysis (price 1 and 9999, huge non-existent ID), state transition (CRUD lifecycle), and error guessing (malformed JSON, missing token).

Q: What is a test plan?

A: The strategy document: scope, approach, entry/exit criteria, environment, risks, deliverables, schedule. Mine is TEST-PLAN.md in the repo.

Q: Severity vs priority?

A: Severity = technical damage; priority = business urgency. A legal-text typo: low severity, high priority. A crash in a never-used feature: high severity, possibly low priority.

Q: What goes in a good bug report?

A: Unique ID, clear title, environment, steps to reproduce, expected vs actual result, severity/priority, and evidence (request/response, report link).

Q: Did your project find real bugs?

A: Yes, two contract deviations: POST /auth returns 200 with 'Bad credentials' instead of 401, and DELETE returns 201 instead of 200/204. Both documented as defects with full reports.

Q: How do you test an API without documentation?

A: Explore: read any code/collection available, ask developers, send safe GET requests, observe responses, and document behavior as I go — then confirm assumptions with the team before asserting them.

Q: How do you decide what to automate?

A: High-value repeatable checks: core flows (CRUD), security rules, and contract checks. One-off exploratory checks stay manual.

Q: What is an SLA in testing?

A: Service Level Agreement — a promised performance limit. I assert every response is under 5000 ms and smoke under 2000 ms.

C. Postman / Newman / CI

Q: What is a Postman collection?

A: A saved, organized set of requests with their test scripts — my whole suite is one collection with 5 folders that run in order.

Q: What are environments and why use them?

A: Variable sets per server (QA/staging/prod). Same collection, different environment file = same tests on a different system, with no hard-coded URLs or credentials.

Q: Variable scopes in Postman?

A: Global (everywhere), collection (my choice for chaining token/bookingId), environment (per server), local (one run). Narrowest scope wins.

Q: Pre-request vs test script?

A: Pre-request runs BEFORE sending (I generate random names/prices there); test script runs AFTER the response (assertions live there).

Q: What language are Postman tests written in?

A: JavaScript, with the Chai assertion library — `pm.test()` defines a test, `pm.expect()` asserts.

Q: How did you chain requests?

A: Test script of `/auth` saves the token: `pm.collectionVariables.set('authToken', json.token)`. Later requests reference `{{authToken}}` in the Cookie header. Same pattern for `bookingId`.

Q: What is Newman?

A: Postman's command-line runner. It executes the same collection headlessly in a terminal — which is what lets the suite run in CI — and produces CLI/HTML/JUnit reports.

Q: Your Newman command?

A: `newman run collection.json -e environment.json -r "cli,html,extra" --reporter-html,extra-export report.html`

Q: What is CI/CD? How did you use it?

A: Continuous Integration = auto-testing every change. My GitHub Actions workflow runs the suite on every push, on PRs, nightly at 2 AM UTC, and on demand — publishing results and an HTML report artifact.

Q: How do you handle test data?

A: Randomized in pre-request scripts so runs are independent, plus a CSV for data-driven iterations covering boundary values.

Q: How do you run data-driven tests in Newman?

A: Add `-d data.csv` — Newman runs one iteration per CSV row, and `{{columnName}}` variables resolve from that row.

Q: What is an auth token and how did you test it?

A: A temporary credential issued after login and attached to later requests — like a movie ticket after showing ID once. I test both sides: with the token, updates succeed; without it, PUT/DELETE must return 403.

Q: Postman vs Newman — when to use which?

A: Postman GUI for building, debugging, and exploring; Newman for repeatable headless runs locally and in CI.

Q: What reports does your pipeline produce?

A: CLI summary, an htmlextra HTML report (uploaded as a CI artifact), and JUnit XML published as a pass/fail check on the commit.

Q: If a test fails in CI, what do you do?

A: Open the run, find WHICH assertion failed, check request vs response in the report, reproduce locally with Newman/Postman, then decide: app bug (file a report) or test/environment issue (fix the test). One known flake: cold-start latency — rerun handles it.

PART 6 — Your Project Cheat-Sheet (read right before viva)

The 60-second project pitch

"I built an automated API test suite for the Restful-Booker booking API. I started with a formal test plan and designed 16 test cases using equivalence partitioning and boundary analysis. In Postman, a smoke test gates the suite, then I generate an auth token and chain it through collection variables into later requests. I cover the full CRUD lifecycle — create, read back to verify data integrity, update, patch, delete, then confirm 404. I validate the response contract with JSON Schema, not just status codes, and attack the API with negative and security tests — wrong credentials, malformed JSON, and mutations without a token, which must return 403. The suite runs headlessly via Newman with HTML reporting and is wired into GitHub Actions: every push, every PR, and nightly. It found two real contract defects, which I documented in formal bug reports."

Project numbers (memorize these)

Metric	Value
Requests in the suite	15
Assertions	41
Test cases documented	16 (docs/TEST-CASES.md)
Folders	5 — Smoke, Auth, CRUD, Negative & Security, Query/Filtering
Defects found & documented	2 (auth 200-instead-of-401; DELETE 201-instead-of-204)
Local run time	~7 seconds
CI triggers	push, pull request, nightly cron (02:00 UTC), manual
Stack	Postman, JavaScript/Chai, Newman, htmlextra, GitHub Actions, JSON Schema, CSV

Last-minute quick scan

If they say...	You say...
API?	Messenger between systems: request in, response out. Waiter analogy.
Methods?	GET read, POST create, PUT full update, PATCH partial, DELETE remove.
200/201/400/401/403/404/500?	OK / Created / bad request / not logged in / not allowed / not found / server bug.
PUT vs PATCH?	Replace everything vs change only sent fields.
401 vs 403?	No ID vs valid ID but not on the guest list.
Idempotent?	Same result no matter how many times: GET, PUT, DELETE. Not POST.
What do you assert?	Status, body, schema, headers, time.
Newman?	Postman's CLI runner — enables headless + CI execution.
Token?	Movie ticket: login once, attach ticket to every protected request.
Chaining?	Save token/ID to a collection variable, reuse as {{variable}}.
Schema test?	Structure blueprint — catches silent field-type changes at status 200.
CI?	GitHub Actions runs the suite on push/PR/nightly, publishes reports.

Final tip: for every answer, add "...and in my project I did this by..." — connecting theory to YOUR repo is what separates a memorized answer from a hire.